

OpenGraph Checker App Summary (Repo Evidence)

Generated from repository code only. Date: 2026-02-03

What it is

Social Sight (repo name: opengraph-checker) is a Next.js app that audits a URL's social metadata and scores OpenGraph readiness.

It also adds growth tooling around analytics tracking, AI-assisted copy/image generation, and paid feature gates.

Who it's for

Primary persona: founders, marketers, and growth teams who want better social preview quality and measurable link performance without heavy setup.

What it does

- Scans URLs via `/api/inspect`, extracts metadata, validates social images, and returns a numeric score with prioritized issues.
- Shows live social preview and score audit UI, including metadata snippets and fix-oriented recommendations.
- Generates AI CTR suggestions (Gemini) from title/description via `/api/suggest`.
- Generates OpenGraph images via `/api/generate-image` and logs generated assets in Supabase (`ai_images` table).
- Compares up to 3 URLs side-by-side in competitor benchmarking with relative win/lose scoring.
- Tracks page views/clicks with a pixel script plus AB/pricing variant attribution and source/referrer parsing.
- Provides authenticated scan history, site verification flow, analytics dashboard views, and Stripe checkout/webhook billing updates.

How it works (architecture)

- Frontend: Next.js App Router with client components (tabs for audit/fix/compare/monitor/analytics/history) and hooks (`useScrape/useProfile`).
- API layer: route handlers under `src/app/api` for `inspect`, `suggest`, image generation, stats summary, tracking, site creation/verification, checkout, and webhooks.
- Data flow: UI -> API routes -> Supabase tables (`scans`, `analytics_events`, `analytics_sites`, `profiles`, `ai_images`, `featured_requests`) -> UI refresh/cache.
- Scraping path: Axios + Cheerio fast path, then Puppeteer + `@sparticuz/chromium` fallback for blocked/time-out targets.
- Identity/payments: Supabase auth (SSR/client helpers + middleware) and Stripe Checkout/Webhook for tier changes.
- Not found in repo: queue workers/background job runner or separate microservice deployment topology docs.

How to run (minimal)

- Install dependencies: `npm install`
- Create `.env.local` with required keys used in code (Supabase URL/anon/service role, Stripe keys/price IDs, `GEMINI_API_KEY`).
- Start dev server: `npm run dev`
- Open `http://localhost:3000`
- Not found in repo: a canonical `.env.example` and explicit local Supabase startup/apply-migrations command sequence.